Experiment with practices from the agile business analysis experts, such as the 7 Product Dimensions and structured conversations, to help your team identify what features to build. Remember to ask questions and to aim for that shared understanding of what your team is building.

MORE TOOLS OR TECHNIQUES FOR EXPLORING EARLY

Eric Ries (Ries, 2010) talks about the mindset needed by companies that are innovating and trying new ideas. The ideas he recommends for lean startups can be used in many situations. Build-measure-learn (BML) is all about building small chunks, measuring, and learning by getting early feedback and validating ideas. Sounds like testing, doesn't it?

Testers can add value in these situations by asking questions and helping to determine what can be measured. Sometimes we can do simple things like review paper mock-ups that programmers or UX designers have created.

A/B testing is one technique that can be used to test a hypothesis. The idea is to develop two separate implementations and put both out for customers to actively use, and then to measure the results. Users are directed to different implementations. The company monitors statistics such as which customers click through or buy something and which leave right away. In this way, companies can decide which experience the customers like better based on actual evidence. One of our contributors has shared his story about A/B testing in Chapter 13, "Other Types of Testing."

As we mentioned in Chapter 7, "Levels of Precision for Planning," visualizing different options with tools such as mind maps is a powerful way to uncover hidden assumptions.

INVEST TO BUILD THE RIGHT THING

The future is unpredictable; we can only make our best educated guess. However, by asking good questions and visualizing possible answers, we can help our customers target the most valuable features to develop next. Start by learning the purpose of each proposed feature. Think ahead to what would happen when the feature is deployed to production. How will we know it's successful? How will we judge whether it

provided the right value? How will we measure or monitor? Answering those questions early can help avoid spending time on the wrong things.

There are many aspects of quality, and we realize that techniques such as the 7 Product Dimensions just scratch the surface. We have to balance functionality with reliability, security, performance, and other attributes valued by our customers. We have to weigh the cost of delaying a release while we work to build a feature that's good enough against the need to delight our customers and make them want to buy our product.

Invest time to help customers identify the most valuable feature to build next, and deploy the minimum viable product. If you can avoid wasting time delivering the wrong thing, your team will have more time for critical testing activities.

In the next chapter, we'll go into more detail about synergies between testing and other skill sets that help us start each new product cycle with a focus on building the right thing.

Summary

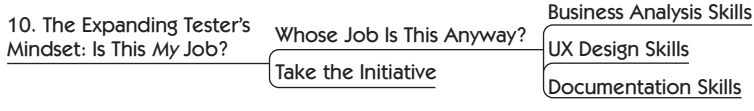
It's hard for customers to know what they want, and even harder for them to explain it to the delivery team. We can help them identify the most valuable features to build and help them articulate what features they want so that we have that shared understanding of what to build. In this chapter, we looked at the following concepts:

- Start with the "why," the purpose of the software we're developing.
- Try out some different tools for helping customers articulate what they want, including
  - Impact mapping
  - Story mapping
  - 7 Product Dimensions
  - Lean startup techniques
- Experiment with different tools that help your development and customer teams collaborate to build the right thing.
- The process needs cycles of modeling, hypothesis, experiments, discovery, and learning, just what a testing mindset provides.

This page intentionally left blank

Chapter 10

The Expanding Tester's Mindset: Is This *My* Job?



In the preceding chapter, we mentioned the overlap between business analysis and testing. Both business analysts and testers work with customers to understand their desires for each feature, but other roles get involved, too. For example, user experience (UX) design experts seek to understand both how people will use the product and the value the business hopes to derive from proposed new capabilities. Technical writers must understand how different users will use the system so they can document how it meets their business or technical needs. However, a team or organization might lack those specialized skills, so when the need arises, team members can learn and use some valuable practices and techniques from those other specialties.

Whose Job Is This Anyway?

As we mention in Chapter 3, "Roles and Competencies," agile development encourages generalizing specialists with T-shaped skills. Nevertheless, we often need team members with specialized training and experience in certain areas to understand business needs.

Business Analysis Skills

We've realized more and more, since writing *Agile Testing*, that BAs bring some important specialized skills to the agile team party. They facilitate structured conversations with the business experts. They remember to ask questions about all dimensions of the product. They understand the whole

business domain, not just the parts that are automated with software. Business analysts historically help business experts communicate their needs to delivery teams. Bernice Niel Ruhland told us about her experiences managing both testers and business analysts. Often the same people did both roles. Having both skill sets on the team worked well because they were always thinking about testing and requirements together.

Like testing experts, BA experts know how to ask good questions and to start with the "whys." Many BAs know how to specify tests based on customer examples and likely have some tools in their toolkit that are beyond the experience of most testers. We need to think about the business problem first, and from a testing perspective, we may focus on the solution too early. That means that, as testers, we may need to think differently when trying out BA skills.

Putting on the BA Hat

> **Mike Talks**, *a tester from New Zealand (Kiwiland), tells us that when he first tried writing requirements, he focused on the solution rather than the business problem.*

Something I really encourage people to do is rally around the Kiwi ideal of "having a go" at something outside their comfort zone within their team. This goes back to the early days of New Zealand as a pioneering colony, where resources and skills were sparse, and generally a culture of generalizing specialists was encouraged. Sound familiar?

One project I was on needed some major up-front business analysis, and I'd tested enough business requirements in my life to think it would be easy. I went around, had my consultations with the business, and wrote everything up. Unfortunately what I documented didn't describe the issues very well and was far too detailed. I probably learned more in a month of doing the BA job than in years of being on the other end of business requirements. It also gave me much more empathy with people in the BA role.

Returning the favor, our project business analyst then helped me with testing and, like me, also trod the crooked path before coming out all right. That glimpse and empathy helped us work more closely together and understand how our jobs fed into each other's work. In my experience of giving the BA role a try, the hardest thing was learning to get the level of required detail right. Focus on the needs, and try not to shoehorn a design solution in—otherwise you're constraining your team's ability to meet those needs.

If your team has testers but not BAs, or vice versa, look for ways to acquire missing skills that might help your team do a better job of learning what to build. Lisa's team, unable to hire a BA, formed a BA community of practice, where team members met to share BA skills learned from books, articles, and conference sessions. They learned better ways to talk with business stakeholders and expanded their perspective to include more dimensions of quality. For the full story, check out Chapter 5, "Technical Awareness."

Considerations on Requirements and the Purpose of Testing

Pete Walen *shares his thoughts about the intertwining of defining requirements and testing.*

It is good to know what the requirements are, what tests exist, which tests are set up, and which ones have been run. However, little of that matters if we do not know the problem the project is intended to address. It is crucial to know the business purpose the system is intended to fulfill.

When you sit down and actually speak with business people in a conversation, you can learn things that may not otherwise be discussed. When your attitude is "Please help me to understand so I can help you better"—as a servant, not a superior—walls come tumbling down, and oftentimes a weird thing happens: people tend to share information freely.

When programmers, designers, or testers come in as the experts who know how to fix problems that business customers have, it can seem condescending. When we approach the problem as colleagues, we can find ways to inform our decisions.

What about Business Analysts?

A good business analyst can gain information and pass it on to the team. However, with each layer introduced in the information flow, the information passed along will be altered. Sometimes information may be lost; other times material may be added. The "clarification" may critically change part of the message. I find that when I get close to the people doing the work, I often learn stuff that is important to me as a tester that other people shrug off as unimportant.

If I can understand the customer's needs better, I can exercise the software more efficiently. If the tests I write and run do not explicitly exercise the requirements, are they really needed? Are they real, or my interpretation of something someone said?

Of course, important ideas will be identified in the documented requirements. There may be other things like, "Don't corrupt the database" or, "This needs to be reflected in the ziggidy-splat system" or, "This value should match what is on this other screen." Do these things make the users' list of expectations? Are the business needs reflected completely in the documented requirements? Only in these conversations do I find the answer.

My advice: exercise both the requirements and the stories. If you can't exercise both, exercise the stories, and then have folks from the business go through the results with you.

UX Design Skills

Companies such as Apple have proven the importance of product design. As Andy Budd has noted (Traynor, 2011), "delightful attractions" such as the swipe to unlock the iPhone "make people fall in love with a product." UX designers are integral, key members of agile software teams today.

Testers and UX designers collaborate productively from the early stages of feature design. Testers, especially those who have direct contact with end users or who use the product themselves, provide useful feedback on UI mock-ups, workflow, and other aspects of design. Good UX designers know how to get value from usability testing and are happy to partner with testers to do those activities. Some activities centered on usability that are common to both UX designers and testers are A/B testing, focus groups, and even paper prototyping.

Planning techniques such as impact mapping can help you better understand the business needs and wants. We don't know how the iPhone was designed, but we can imagine an example impact map for it:

- **Why (Goal):** Gain a market share of 70%
- **Who (People):** Prospective and existing iPhone users
- **How (Behaviors):** Easy phone unlock that users fall in love with
- **What (Activity/Feature):** Design swipe gesture to unlock

The team (including testers, of course) could then brainstorm other Behaviors and Activities that might meet the goal.

Lisa's Story

Our team had one UX designer who worked remotely and was spread much too thin in our team of around 20 developers. When we hired a second UX designer in our Denver office, new opportunities opened up for us testers.

Another tester on my team had the idea to set up a brainstorming meeting with all three testers and both designers. The topic was "visibility of design prior to production."

We've been trying some of the resulting action ideas, which included:

- Go over design features for stories together. Note subsequent changes (for example, due to implementation constraints) in story comments.
- Designers pair with testers for accepting stories that include implementing feature design.
- Designers involve testers more in testing interaction and flow design choices.

These experiments have helped reduce time wasted in reworking user interface (UI) design after coding starts. Acceptance testing goes more smoothly when we pair since the UX designers can spot implementation issues such as an incorrect font or color right away. Even when we don't pair, designers are available to answer questions immediately, so we are never blocked.

We testers helped the designers refine their usability testing script. Thanks to our designers' expertise and experience with usability testing, we received useful feedback from our product's users early in the development process. Testing our new UI design was easier with help from the designers. Once our new UI design was released in beta, customer feedback was overwhelmingly positive.

Even if you don't have specialists in things like UX design and usability testing on your team, you can learn enough to get valuable feedback from users. See Chapter 13, "Other Types of Testing," for examples of usability testing.

Documentation Skills

Many teams still have the need for user documentation, whether it is electronic or paper based. If you are on one of the fortunate teams that are able to work closely with technical writers, you probably already know how valuable they can be. In *Agile Testing* (pp. 208–9), we gave some examples based on our experiences. Technical writers are in a unique position to challenge ideas; when you have to articulate a concept, it sometimes doesn't translate as well as you think.

When you do not have a technical writer on your team but still are responsible for user documentation, consider adding the task "Create content" to each story. This task can be completed by anyone on the team. At the feature level, there can be a task to "Complete user documentation." By then, the user interface is stable, so if you need to take screen shots, you can. The content can then be assembled and edited by anyone on the team, or perhaps by the company technical writer.

Another idea is to pair with others on your team. For example, Lisa paired, not with a technical writer, but with the UX designers on her team when testing extensive online documentation for their API.

TAKE THE INITIATIVE

The joy of blurred roles on agile teams is that it makes our jobs more interesting! If you sense that your team and its customers are struggling to define a feature, put on another hat. What practices from a different profession can you try?

Lisa's Story

I worked for many years on a team producing financial services software. Our software managed all aspects of 401(k) retirement plans. The system was originally modeled so that only one person from each employer with a 401(k) plan could log in and administer that employer's plan. This "plan sponsor" did activities such as submitting payroll contributions, enrolling new employees in the plan, and approving distributions and loans from 401(k) accounts.

The "plan sponsor" term was used all over our website. At one point, we started a feature to allow multiple user accounts to do "plan sponsor" activities. However, by law, the actual plan sponsor is the employer. We had a number of small-group discussions about terminology. Everyone had a different opinion.

I finally scheduled a meeting with all the stakeholders to make a final decision on terminology. Is getting people together like this a tester's job? Why not? Our ScrumMaster happened to be out sick, and our product owner

was always swamped. Development on the feature had started, and we were wasting time with indecision over verbiage.

In 15 minutes, the various business managers agreed on the term "plan administrator" for all the users from a given employer who would log in to do what used to be termed "plan sponsor" activities. There would still be just one plan sponsor per plan whose name would be used in all the legal documents.

Since the product owner was too busy to do it, I mocked up all the changes for the many pages on the site that, up to now, had said "Plan Sponsor" (see Figure 10-1). I posted them on the wiki so our remote programmer (who works in India) could see them and added a task card to the relevant story row on our board. By the next morning, our remote programmer had completed most of the changes, and another programmer completed the rest. I verified the changes, showed the customers, and updated an automated regression test.

Figure 10-1 Simple mock-up

Yes, it would have been preferable to get the terminology worked out prior to starting the story. After this and other similar experiences, we started pushing back on the business to have all the business rules and examples ready by the start of the iteration. If we lacked the necessary information, we postponed the story to the next iteration. We know questions and changes during development are inevitable. But whenever we can find ways to move forward and save time and improve quality, whoever has the initiative can lead the way!

Many of the skills that business analysts and UX designers practice are really what we call 'testing for business value.' Challenging ideas, and testing those ideas and assumptions, moves us closer to the features that our customers value. In the next chapter, we will talk about how to get examples to express a shared understanding. It is about developing the right "thing."

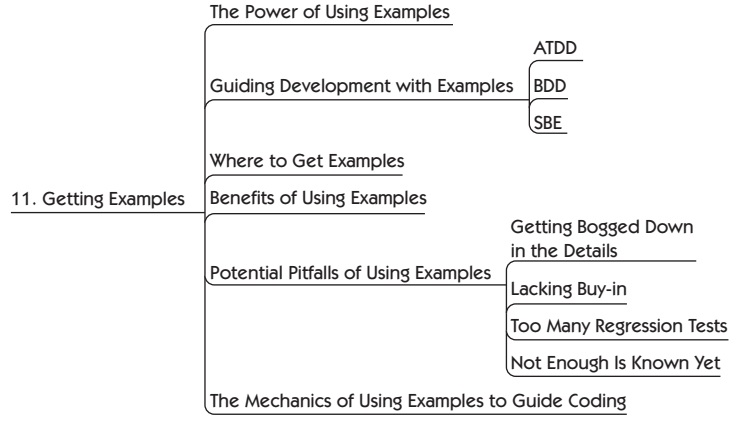
SUMMARY

Blurring the distinctions between roles can make our jobs so much more interesting. This chapter was about expanding your mindset and learning from others.

- Brainstorm with the team to identify what skills may be missing.
- Try creative experiments; add to your T-shaped skills to fill skill gaps. Wear a different specialty "hat" when needed.
- Business analysts' skills are synergistic with those of testers. If your team has both testers and BAs, they can work together for better software quality.
- Testers and UX designers can partner for activities such as usability testing, prototyping, and getting fast feedback from users. This helps the team save time and stay on track.
- All team members can put on a technical writer's hat if necessary and create content, think about how a concept might be translated into words, and perhaps even edit and test user and technical documentation.
- Learn practices from other professions, such as business analysis and UX design, to help build multiple dimensions of quality into software features.
- When you see a problem, take the initiative to get the right people together and discuss possible solutions.

Chapter 11

Getting Examples



In our experience, eliciting examples from customers is a powerful way to guide development with both the technology and business-facing tests. Examples form the heart of Quadrants 1 and 2, whether it's a paper prototype, a flow diagram drawn on the whiteboard, or a spreadsheet with inputs and expected outputs. In *Agile Testing*, we talked about examples in just about every chapter, and that's true in this book as well. One of our favorite questions we ask if we're not sure about what is under discussion is, "Can you give me an example of that?" We suggest you do the same.

The Power of Using Examples

We first learned about example-driven development from Brian Marick back in 2003. As we explained in *Agile Testing* (pp. 378–80), we've depended on working with customers to get examples of desired and undesired system behavior so that we know the right things to build for our customers.

Examples in Real Life

The following story from **Sherry Heinze**, *a tester from Canada, shows how we can use examples in our everyday life.*

I was working at a client site that closed for four full days and two partial days over the holidays. Management sent out the holiday schedule in a table format. One manager who had both staff and contractors sent out a copy of the schedule with a note to help people book their time off correctly. *Note:* The company's regular schedule was 7.5 hours daily, Monday through Thursday, and 7 hours on Friday.

> *When entering time over the holidays, please look at the charts [see Figure 11-1] below. For office closures (other than Stat), please book your time to this Admin code, otherwise your time will be entered as Vacation or against standard billable codes.*
>
> *If you are a contractor, please only enter time in the office and do not use the above Admin code. On days & times the office is closed, the time entered for contractors should be 0.*

	If on Vacation			If Not on Vacation		
Date	**Vacation**	**Stat Holiday**	**Admin**	**Regular Billing**	**Stat Holiday**	**Admin**
Monday Dec 23 (Regular Office Hours)	7.5	0	0	7.5	0	0
Tuesday Dec 24 (Closed at 2:00)	4.5	0	3	4.5	0	3
Wednesday Dec 25 (Closed – Stat)	0	7.5	0	0	7.5	0
Thursday Dec 26 (Closed – Stat)	0	7.5	0	0	7.5	0
Friday Dec 27 (Closed)	0	0	7	0	0	7

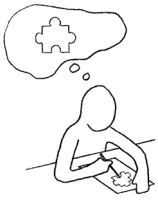
Figure 11-1 Time chart

I looked at this chart and then asked some clarifying questions since I start work at 7:00 a.m. and am a contractor.

- Does this mean I can't bill for the 6 hours I normally would have on December 24?
- If not, should I leave at 11:30 to keep the 4.5 hours?
- Do you want all of us to come in at 9:00 on December 24?

- Does this mean I can't work at home that day, as I usually do if the schools are closed?
- What if I am an employee who works in a support group and must start earlier to answer the phones?

What I found out was that the person who created this chart assumed that everyone else started work at 9:00 a.m. and worked only in the office.



Sherry's story shows us how an example as simple as a chart can give us a starting place to ask concrete questions, working toward a shared understanding of a feature's goal.

Functional test tools have matured over the last few years to help us capture examples and turn them into automated tests. This is a good thing, but it's possible to automate lots of tests and still fail to deliver what the customer really wants.

In his Agile Testing Days 2013 keynote, J. B. Rainsberger noted that many teams still do not take advantage of the simple idea of "talking in examples" (Rainsberger, 2013). His message echoes Liz Keogh's (Keogh, 2012a):

> **Having** conversations
>
> is more important than **capturing** conversations
>
> is more important than **automating** conversations

Likewise, many teams use a business-readable domain-specific language (DSL), often in a given_when_then format, to describe feature behavior without getting into implementation details. You can use this technique in automated test scripts, but it may have more value as an analytical tool. Collaborating with customers to create given_when_then-style examples helps define useful, appropriately sized stories that, when finished, will provide the desired feature. Automation becomes a useful side effect of using the tool.

If your team is not already working with stakeholders to get examples of desired and undesired system behavior, start experimenting with

this approach. See if it helps you to understand more quickly what your customers want and to deliver the right things with less churn and wasted time. Janet has found this to be a useful rule of thumb: if you have more than one example of desired behavior, you might need more than one story.

We decided to focus a whole chapter in this book on getting examples because it is such a key practice and still hasn't caught on with many teams. Unfortunately, the name "example-driven development" never caught on either. The common names currently being used to describe the process are

- Acceptance-test-driven development (ATDD)
- Behavior-driven development (BDD)
- Specification by example (SBE)

In *Agile Testing* (pp. 99–101), we referred to these tests as business-facing or customer-facing tests that drive development in a similar fashion to test-driven development (TDD), but at the business level. They help define external quality by defining the features that customers want. In this book, we refer to the practice generically as "guiding development with examples." Feel free to exchange the term for whichever one you use on your team. We attempt to explain the differences here, but in reality they are all about starting with examples and conversations.

The choice of tool may determine which practice you adopt, so we advise that you know what you want before you choose a tool. For example, in workflow-type applications, describing the behavior may be applicable. If the application is calculation heavy, a spreadsheet or tabular test format with specific examples might be more appropriate. We give examples of both formats in Chapter 16, "Test Automation Design Patterns and Approaches."

GUIDING DEVELOPMENT WITH EXAMPLES

Don't get distracted by jargon or buzzwords. Whether you choose to call your process ATDD or BDD or SBE, the goal is the same—a common understanding to build the right thing the first time. All of them follow a flow similar to the one shown Figure 11-2 with slight variations in the

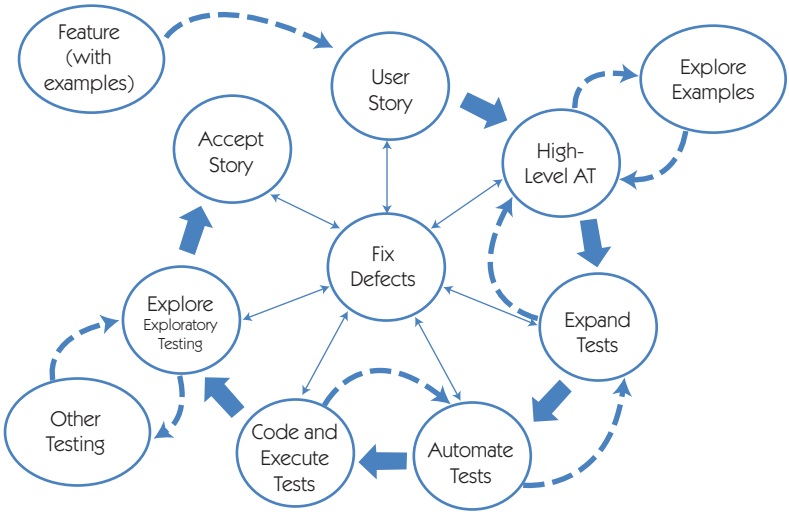


Figure 11-2 Guiding development with examples

naming. The most important thing is to remember to start with the end in mind.

ATDD

Jennitta Andrea has described ATDD as (Andrea, 2010)

> *. . . the practice of expressing functional story requirements as concrete examples or expectations prior to story development. During story development, a collaborative workflow occurs in which: examples are written and then automated; granular automated unit tests are developed; and the system code is written and integrated with the rest of the running, tested software. The story is "done"—deemed ready for exploratory and other testing—when these scope-defining automated checks pass.*

One problem with the label "acceptance-test-driven development" is that "acceptance test" is a vague term that means different things to different people. It can be confused with user acceptance testing (UAT),

where an end user or outside vendor "accepts" the product. This acceptance may be tied to contractual payments.

We define acceptance tests as the tests that describe the business intent each story must deliver. Depending on how your team implements the practice, the acceptance tests may include only the high-level expected behavior and a couple of examples of misbehavior. However, they may also include a broad range of tests that encompass everything except TDD at the unit and component level. They may include quality attributes such as usability and performance, although we prefer to think of those as constraints that apply to all stories. Most people refer to functional tests when they talk about ATDD.

Delivering Value through Conversations at Paddy Power

> **Augusto Evangelisti**, *a software development professional in Ireland, shares his experiences with conversations in his organization.*

I joined Paddy Power as a principal test engineer in 2012, and my task was clear from the very first day I met my boss. In fact, on that first day he said to me something along the lines of "Gus, you need to improve quality without negatively impacting throughput." Even though the remit was a challenging one, I had been in similar circumstances before, and I wasn't really scared (silly me).

I set about observing the teams so I could understand where improvements might be made. The teams were already composed of excellent developers who followed very good practices. They had very high unit test coverage, conducted code reviews, and pair programmed, and they had a really good continuous integration (CI) system.

They even did acceptance-test-driven development (ATDD). Now I was scared! I kept observing and started digging deeper to find what was missing. The first concrete suggestion I made was to embed the business analysts into the teams. They had been operating as a separate team up until that point. It was a small change that helped a little, but I was still at a loss as to what else might be going on.

A few more weeks went by, and slowly but surely I started to understand where the problem might lie and why we had quite a

few incidents in UAT and even in production. The key was observing how people practiced acceptance-test-driven development. On the kanban board, there was a column called "Awaiting BA Sign-off," and after that, development began. Essentially the user stories were handed over from the business analysts (BA) to the developers through a formal sign-off, at which time the analysts moved on to begin work on the next set of user stories. There were no conversations. The developers would take the user stories and transform them into what they thought were the acceptance tests and corresponding code.

I was very lucky to find a great ally in Mary, another test engineer who shared the same desire to make things better. We worked together to size the problem, and we used an impact-mapping session to identify a possible solution. The approach we identified was to create and facilitate workshops on "real" ATDD with the development teams.

We tailored our ATDD approach to our specific context, using some of Elisabeth Hendrickson's ATDD approach and Gojko Adzic's specification by example. The key was emphasizing the importance of

Conversations over automation

We told the teams: ATDD is about people, communication, collaboration, and delivering business value.

We moved all the emphasis to this aspect and explained how the automated regression suite that is created during the process is no more than a natural outcome. The value is in collaboratively defining the user stories, sharing the understanding of the application, and delivering the business value to our customers.

To collaboratively define the user stories, we removed the "Awaiting BA Sign-off" column from the board and replaced it with a column called "Discuss," where the "3 Amigos" sit together, talk about the user story, and identify examples that will later become acceptance tests and, in turn, code and eventually business value. I thought this was a perfect example of

Customer collaboration over contract negotiation

The workshop approach really helped to trigger change in the team's behavior, and over a matter of weeks the development team members were implementing the new approach, with immediate and tangible impact.

After one year, the results have been amazing. The issues in UAT are practically nonexistent, and the same applies to production. The teams really understand the product they are building, and the business analysts, test engineers, and developers work as one strong, single unit. Last, but not least, we even improved throughput by removing the rework of issues as a result of poor quality. The misunderstandings that used to surface at the UAT stage are now identified and resolved before a line of code is written.

Ah, I almost forgot: we also have a lot of fun!

BDD

Behavior-driven development (BDD) uses natural language to capture customer examples in a domain-specific language. The goal is to have those important customer conversations and create given_when_then scenarios that express the behavior of a feature, including the expectations for a particular condition and action, in tests that everyone on the team understands.

Dan North's description of BDD (North, 2006) is a bit more complicated, but we interpret it as using the word *behavior* rather than *test*, and using the given_when_then guidelines to describe the precondition, trigger, and post-condition of what is expected—before coding starts. In a post to the Yahoo Agile Testing group in August 2010, Liz Keogh explained (Keogh, 2010):

> *BDD's focus is on the discovery of stuff we didn't know about, particularly around the contexts in which scenarios or examples take place. This is where using words like "should" and "behavior" comes in, rather than "test"—because for most people "test" presupposes that we know what the behavior ought to be. "Should" lets us ask, "Should it? Really? Is there a context which we're missing in which it behaves differently?"*

Originally a response to TDD, BDD has evolved into both analysis and automated testing at the acceptance level. Feature Injection is a business analysis process framework that focuses BDD and TDD on delivering business value (Matts and Adzic, 2011). Teams using Feature Injection start by defining and communicating business value, create a list of features to drive delivery of that business value, and then expand the scope of those features, investigating high-level examples of customer use. We include examples of BDD-style tests later in this chapter.

SBE

As a result of the Agile Alliance Functional Test Tools group (AA-FTT) workshop at Agile 2010, Declan Whelan, Gojko Adzic, and a few others came up with a diagram (Figure 11-3) to try to get a common understanding around specification by example.

Look closely at the diagram. You may notice that there is not a single mention of testing. SBE starts with the goal in mind to derive the scope. This is where techniques such as impact mapping give us a great start by first identifying the right goals. To get to key examples that explain the context sufficiently, the team specifies collaboratively, perhaps in a specification workshop or similar activity. These key examples become our high-level acceptance tests. We refine those examples, extracting a minimal set to specify a business rule adequately; these become the story tests. At this point, we automate without changing the meaning, and those automated tests become the checks to give fast feedback on a regular basis. The result: executable specifications or living documentation that describes the functionality of the system at any point in time.

Gojko Adzic has explained how teams can use specification workshops to create examples, getting the right people to collaborate to get a good understanding of what to build (Adzic, 2009). See the bibliography for Part IV, "Testing Business Value," for more links to learn about SBE.

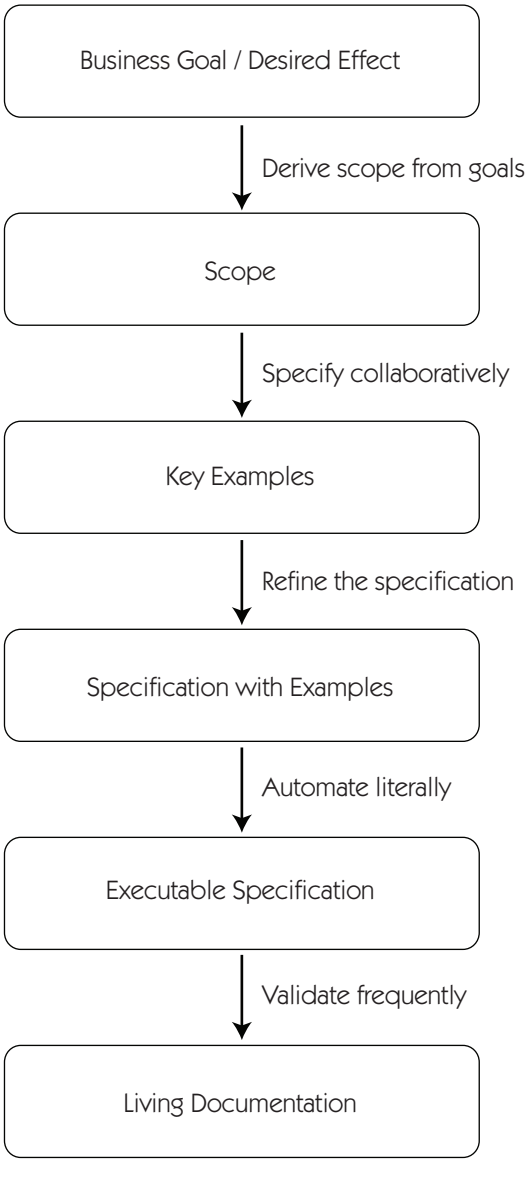


Figure 11-3 Specification by example (Adzic et al., 2010)

WHERE TO GET EXAMPLES

Whatever we call the process, we want to minimize rework and misunderstandings of what to build. In *Agile Testing* (pp. 378–80), we gave some simple ways to collaborate with customers to specify examples. We'll explore a few others here. The best way to get examples of how a particular feature or story should work is to sit with your customers and watch as they do their jobs. Try to imagine where the new feature will help or hinder them.

Informal brainstorming sessions where participants draw on whiteboards (physical or virtual) usually work well. More formal specification workshops are another option; however, getting the right people to participate is key (Adzic, 2009).

Examples can be elicited as part of an impact-mapping session, and they naturally emerge in story-mapping workshops. See Chapter 9, "Are We Building the Right Thing?," for details on these practices. Whatever you use for the format of your meeting to obtain examples, set a time limit, and keep the meeting short and focused. People can stay fresh during an hour-long meeting, but if you need more time, schedule another session for a few days later. That gives everyone time to ruminate on what they want and come up with useful examples.

Support tickets and community forums can be sources of examples of what people need to do with your product. The frequency with which a particular feature is requested indicates what's most important to users.

Lisa's Story

We decided to upgrade our product's search engine. As this involved reindexing everything in our database, it was a good opportunity to add some search functionality. We didn't have a lot of time to spend, but we could go for the "low-hanging fruit."

Our testing and customer support manager already had a good sense of what customers wanted. She looked through feature requests and complaints on our community forum as well as past customer support tickets and listed the most-requested search functionalities. Our delivery team was able to add a surprising number of them. It was a big win that really pleased our customers.

Ellen Gottesdiener and Mary Gorman use the template shown in Figure 11-4 to get examples in a given_when_then format (Gottesdiener and Gorman, 2012). It ties the story to the scenario, along with its data. It also makes the writer consider some of the dimensions we talked

Story	As a supervisor, I can search in the employee database so that I find a specific employee's information.
Scenario	Valid name search returns results.
Business rule	Search returns only employee names who report to the requesting supervisor.
Given	
Precondition(s), state	A supervisor exists with direct reports.
Fixed data	Supervisor: Kant Employee Last Name: Smith
When	
Action	Kant navigates to the employee name search page and enters the value "S."
Input data	Search criteria – "S"
Then	
Observable outcome: Message, output data	Kant sees a search result that includes Smith.
Post-condition(s), state	Smith is displayed in search result.

Figure 11-4 Given_when_then template (Gottesdiener and Gorman, 2012)

about in Chapter 9, "Are We Building the Right Thing?" Not all stories need a template like this, but it helps draw out the ideas in new areas of the code.

Use business rules, use cases, business expertise (customers, business analysts, domain experts, etc.), the expertise of the delivery team (programmers, testers, DBAs, etc.), along with any other supporting information you might have to get examples.

Make time to learn your business domain as much as is practical, so you can help stakeholders create and articulate useful examples. This knowledge enables you to zero in on the purpose of each feature and strip out unneeded functionality or simplify the approach.

BENEFITS OF USING EXAMPLES

We can get examples of desired system behavior from everyone with a stake in the solution. This is especially important for internal stakeholders such as those in the accounting and legal departments, who are often overlooked in the requirements-gathering process. Thinking in terms of concrete examples helps business experts articulate business rules and engages stakeholders more fully in the development process. If possible, sit down with a business expert, observe how they do their work, and create examples together.

Concrete examples make it much easier for the delivery team, including programmers, testers, operations, user experience designers, and database experts, to understand the needs of the different customers. Rather than having a philosophical discussion of what the best implementation might be, people draw on whiteboards and fill spreadsheets with example inputs and expected outputs to express business rules in terms everyone can understand. We bring all these examples together and turn them into tests to define system behavior, thereby reducing the uncertainty about what needs to be built.

A DSL is one way customers can express their examples in a way that they and the development team can use. A given_when_then style like the examples earlier in this chapter may suit your organization. If you're testing calculations and algorithms, you might prefer a tabular approach, such as a spreadsheet with defined inputs and expected

outputs. Many test frameworks allow examples in a DSL format to be automated directly so they not only are readable by non-programmer team members but can also be executed as part of a continuous integration process.

Programmers can use the DSL examples to help with TDD as well, although TDD is as much about designing the code as creating the unit tests. The broad and deep understanding of the feature that examples provide helps programmers know what code to write.

Preventing Defects with Examples

Cory Maksymchuk, *a programmer in Winnipeg, Canada, explains how test-first coding helps cover a range of example scenarios and prevents defects.*

I was once working on a section of a screen with a dynamically generated drop-down "select box." The contents of the select box would be different based on which user from which organization was viewing it. Although there were quite a few scenarios for what it should do, I had a good handle on them in my head and in the requirements document. At the time, we had quite a few developers and were low on analyst and tester availability, so I was working ahead of our test cases. I finished the code, complete with unit tests, sent it for code review, and committed it. A day later I received the test cases, of which there were 52. My work passed 14 of them and failed the other 38.

Had I only worked through a requirements document and my code made it to production, there would have been a defect log with a pile of work in it. By writing test cases first, we make sure all cases are covered before committing code. Without defects making it to production or even being seen by a business user, we increase confidence, eliminate months of rework and maintenance, and look like stars. After we recommitted to our test-first strategy, this project went live seven months later, on time and on budget. Four developers, two analysts, one tester, and a lead worked on it, and a year later we have received only one small defect report.

Test-first helps us cover all viable scenarios in a way that the users can read and understand. It allows us to understand what "done" means and to not waste our time writing code that is not covered by tests. Couple this with good coding style, thorough code reviews, and good coverage in unit and integration testing, and you are on your way to a no-defects world.

Potential Pitfalls of Using Examples

As with any testing practice, there are risks to guiding development with examples. As your team learns to collaborate with customers to elicit examples and turn those examples into potentially executable tests, be aware of the ways this can go wrong.

Getting Bogged Down in the Details

Gathering any type of requirements before coding begins is a slippery slope toward a requirements phase. When you provide customers with an easy way to create examples, they may tend to get carried away. Detail-oriented business experts also may be overeager and provide every edge case they can think of. When programmers start working on the feature or story, they may be so overwhelmed with detail they can't see the main purpose of what they need to code.

Needing a large number of examples, or extremely complicated examples, may be a sign of a deeper problem, such as incorrect software design or inadequate models. If examples seem too complicated, there may be a flaw or some fundamental gap in the design or the domain model. The more complexity there is in a feature, the more conversation there needs to be to reduce the uncertainty.

Experiment with how much detail is appropriate for your team. If you're planning a feature similar to work your team has done before, you may not need to spend much time discussing examples. If your customers have prioritized a big, risky feature set, hold a time-boxed meeting a few weeks in advance of when development will start, with both development and customer teams, to start discussing examples. If you need more time, schedule another meeting.

In our experience, it works best to limit examples to a few happy path, sad path, and ugly path scenarios when working this far in advance. In story readiness meetings before the iteration, go through the stories again with the business experts to ensure a shared understanding of the feature. When coding begins on a story, it's time to get into the pickier details and make sure all viable cases are covered. Programmers can start by writing code to make the happy path tests pass, then move into the boundary conditions and negative and edge cases. In *Agile Testing*,

we called this "working with steel threads"—work on the core, and then add complexity.

Chapters 8 and 18 of *Agile Testing* gave more information on the appropriate level of detail for examples throughout the incremental and iterative development process. Each situation is unique, so use retrospectives to see if tests based on examples are providing the right amount of information at different times in the coding and testing cycle.

Lacking Buy-in

Customers and delivery teams need to collaborate to get examples and use those to guide coding. This requires buy-in on all sides. Business stakeholders may feel they're too busy or that it's someone else's job to create requirements for features. We have to show them what's in it for them—what the benefits are.

Start by proposing an experiment. Explain the benefits of using examples, and suggest trying them for a new feature that's going to be developed soon. Schedule short meetings to create examples, and don't run over time. If you don't finish, schedule another meeting in a few days. Encourage customers to draw flow diagrams, mind maps, or other visuals as they think of examples. Use formats that are appropriate to the domain, such as spreadsheets for financial applications.

As you discuss more detailed examples, show business experts how you turn appropriate ones into executable tests in the agreed-upon DSL. Demo the resulting software early and often. Even seeing only the happy path behavior work will help customers feel their time is well invested.

Programmer buy-in can also be tricky. Programmers might feel they don't have time to bother with ATDD, especially if they're already doing TDD at the unit level. If the DSL used for acceptance-level examples is different from the format of their unit tests, it might mean too much task switching for their comfort level.

Testers might even need convincing if they are used to defining everything abstractly, such as, "Verify the name is valid," rather than using an example to say that, "Janet Gregory is an example of a valid name, but Janet#Gregory is not." Be prepared to iterate through different

approaches to express and automate examples before finding the one that works for your team.

Too Many Regression Tests

Today's test frameworks make writing test cases in a DSL easy and fast. You can even use them for automated exploratory testing, especially at the API or service level, cranking through a wide variety of inputs and trying all the crazy edge cases. This is great, but beware of keeping all these executable tests in your automated regression suites. Automated regression tests provide a safety net, but they also need to give fast feedback, and we can't spend more time maintaining them than the value they deliver warrants. Once a test is passing, evaluate whether it adds enough value to include it in a regression suite. If it tests an edge case that, if it failed, would be low risk, you might prefer to leave it out or put it in a suite that runs only once per day or right before release. If unit tests cover the same area of the code fairly well, you may not need the higher-level test unless you need it for business visibility.

It's a trade-off between risk and cost. Over eight years, Lisa's previous team had two regression failures in production that required rolling back the release. They had created automated tests that would have caught both of the regressions, but the tests weren't in the suites that ran in the CI. They had made a conscious decision to leave them out, thinking they were edge cases that would never happen. On the one hand, this seems bad, but really, this was only two failures in eight years. That's offset by a lower test maintenance cost by having fewer automated regression tests. This is another area where lots of experimenting is needed to find the right balance.

Not Enough Is Known Yet

If your team starts on a new feature or capability that you've never done before, and nobody on the team has experience in that area, it may be too early to specify concrete examples. As Liz Keogh writes (Keogh, 2012a):

> *When you start writing tests, or having discussions, and the requirements begin changing underneath you because of what you discover as a result, that's complex. You can look back at what you end up with and understand that it's much better, but you can't come up with it to start*

with, nor can you define what "better" will look like and try to reach it. It emerges as you work.

When there are too many unknowns, or you don't even yet know what you don't know, it's more useful to spike solutions and get quick feedback to see if you're going in the right direction. Programmer-tester pairing to explore potential behavior may be useful at this time. The product ideas and technical implementations have to emerge.

THE MECHANICS OF USING EXAMPLES TO GUIDE CODING

Example-driven development, whether you use ATDD, BDD, or SBE, is a core practice for agile teams, and it takes work and practice to learn how to do it effectively. Please see the bibliography for Part IV for books and other resources that teach ATDD/BDD/SBE through examples.

SUMMARY

For years now, we've followed this motto coined by Brian Marick: "An example would be handy right about now." If you find yourself embroiled in an unending team discussion about how a particular feature should work, grab a marker and a whiteboard or flip chart (or the virtual equivalent) and start asking for concrete examples. You'll quickly make progress toward defining what that feature will look and act like once it's "done." Key points in this chapter are

- The power of using examples
- Overviews of the popular approaches to guiding development with business-facing examples:
  - Acceptance-test-driven development (ATDD)
  - Behavior-driven development (BDD)
  - Specification by example (SBE)
- Where and how to get examples from business experts
- Benefits of using examples
- Potential pitfalls of guiding coding with examples
- The mechanics of example-driven development

Part V

Investigative Testing

In Part IV, "Testing Business Value," we looked at ways to help customers set goals that bring value to the business, expressed examples of how software will help achieve those goals, and turned those examples into tests that guide development. These activities give us confidence that we can build the right software for our business. However, we can't know for sure that the testing we do based on the initial ideas about how features should behave will actually ensure that all our customers' needs are met. We need to learn more about the features as we build them, working in a series of small experiments and building in short feedback loops to refine requirements.

As programmers commit new and updated code, we look at the product to see if it meets expectations based on the experience of the people testing it. We investigate the results of each experiment to inform the next. Exploratory testing is one way to investigate. Because agile teams deliver new code frequently, they may tend to focus on functional testing. However, there are many other kinds of tests that question the code and how the team interpreted the business needs. In Part V, we'll consider a variety of quality attributes that agile teams may need to investigate as code is completed.